

BÀI 3: KIỂM THỬ LỚP TƯƠNG ĐƯƠNG

Mục tiêu:

- **Về kiến thức:** cung cấp cho sinh viên kiến thức tổng quan về kiểm thử chức năng, kiến thức về kỹ thuật kiểm thử lớp tương đương: phân loại kiểm thử lớp tương đương, các bước thiết kế testcase.
- **Về kỹ năng:** giúp sinh viên áp dụng kỹ thuật kiểm thử lớp tương đương thiết kế được hệ thống testcase cho bài toán cụ thể.
- **3.1 Tổng quan**
- Kiểm thử hàm (functional testing) là các hoạt động kiểm tra chương trình dựa trên tài liệu mô tả chức năng, yêu cầu phần mềm, hay còn gọi là *đặc tả chức năng* (functional specification). Đặc tả chức năng là tài liệu mô tả yêu cầu, hành vi của chương trình mong muốn. Tài liệu này là cơ sở để chúng ta tiến hành xây dựng các ca kiểm thử, thuật ngữ chuyên môn gọi là *thiết kế kiểm thử* (test design) và người thực hiện việc này là người thiết kế kiểm thử. Thiết kế kiểm thử là hoạt động chính trong kiểm thử hàm.
- Cần nhấn mạnh rằng kiểm thử hàm là việc thiết kế các ca kiểm thử hàm của chương trình chỉ dựa trên đặc tả của chương trình mà không dựa trên việc phân tích mã nguồn của chương trình. Kiểm thử hàm còn được gọi với tên khác chính xác hơn là kiểm thử dựa trên đặc tả, hay kiểm thử hộp đen, hay kiểm thử chức năng. Còn việc thiết kế kiểm thử có dựa vào mã nguồn của chương trình được gọi là kiểm thử cấu trúc, hay kiểm thử hộp trắng.
- Kiểm thử hàm là một kỹ thuật cơ bản giúp phát hiện nhiều lỗi mà kiểm thử hộp trắng không phát hiện được. Ví dụ dễ thấy nhất là các lỗi do cài đặt (implementation) không đầy đủ so với đặc tả. Nếu phần mềm thiếu hẳn một chức năng thì chúng ta dễ nhận thấy, nhưng nếu phần mềm thiếu một trường hợp của một chức năng thì việc này không dễ phát hiện nếu không kiểm tra kỹ chương trình so với đặc tả.
- Tài liệu đặc tả dùng để thiết kế các ca kiểm thử có thể đơn giản là mô tả về chức năng của chương trình viết bằng ngôn ngữ của người sử dụng, nhưng thường

là cả các tài liệu phân tích, thiết kế ở các mức độ chi tiết hơn với các hình vẽ, bảng biểu, sơ đồ làm rõ hành vi của chương trình vì chúng mô tả sát hơn các tính chất hàm của chương trình mà chúng ta sẽ dựa vào đó để thiết kế kiểm thử. Chúng ta gọi chung các tài liệu này là đặc tả chức năng.

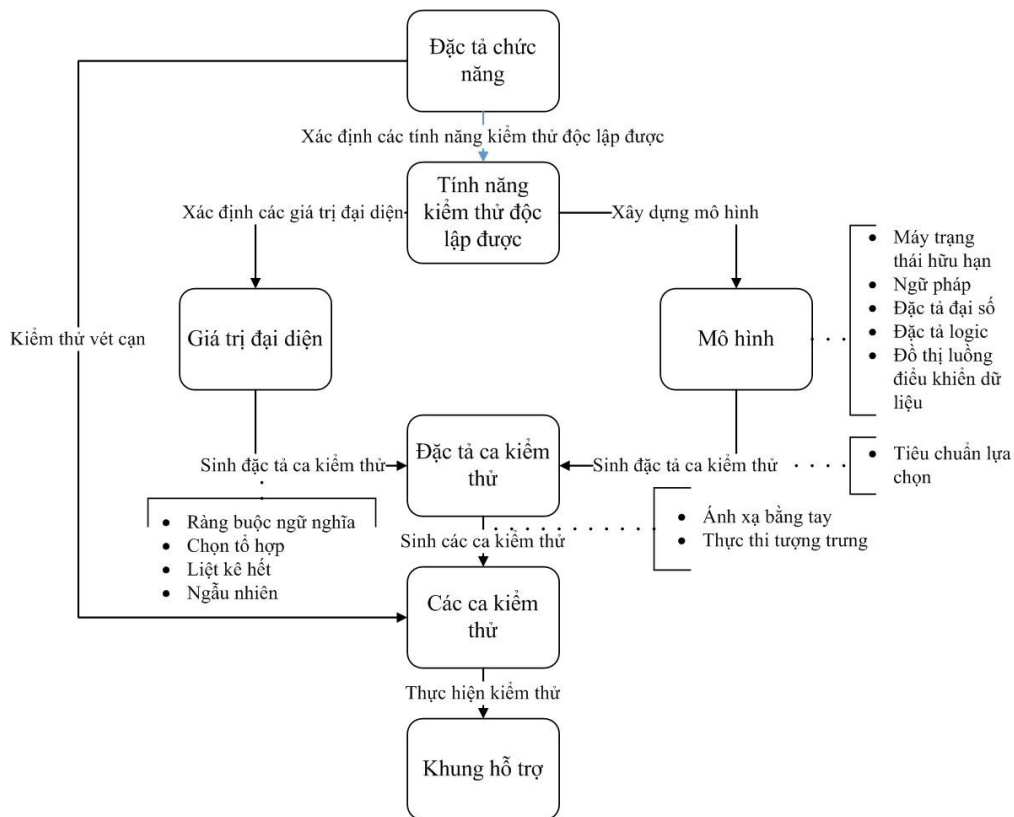
- Dễ nhận thấy do chỉ dựa trên đặc tả chức năng, kiểm thử hàm có ưu điểm là có thể thực hiện được sớm, trước khi cài đặt chương trình. Thời điểm tốt để chúng ta bắt đầu thiết kế kiểm thử là khi tài liệu đặc tả đã ổn định, nhưng cũng có nhiều qui trình phần mềm hiện đại đang đẩy việc này lên sớm hơn, ngay trong giai đoạn xác định yêu cầu đã lồng ghép xác định các ca kiểm thử. Các ca kiểm thử này thường được mô tả dưới dạng các ví dụ, vừa có tác dụng giải thích, làm rõ tài liệu đặc tả, vừa có tác dụng sử dụng luôn làm các kịch bản kiểm thử, các ca kiểm thử.
- Thông thường kết quả của việc thiết kế kiểm thử hàm sẽ đưa ra *đặc tả ca kiểm thử*. Tương tự như khái niệm đặc tả chương trình là mô tả chương trình mong muốn, đặc tả kiểm thử mô tả các ca kiểm thử cần được tạo ra. Từ một đặc tả ca kiểm thử chúng ta có thể tạo ra nhiều ca kiểm thử cụ thể để tiến hành kiểm tra với chương trình. Ví dụ với một hàm giải phương trình bậc hai với đầu vào là ba hệ số là a, b, c , đặc tả ca kiểm thử cho nghiệm kép là với mọi a, b, c thỏa mãn $b^2 = 4ac$. Từ đặc tả này ta có thể tạo ra các ca kiểm thử cụ thể, ví dụ $a = 1, b = 2, c = 1$, và kết quả mong đợi là nghiệm kép -1 .
- Bản chất của các đặc tả ca kiểm thử là mỗi đặc tả ca kiểm thử sẽ xác định một lớp hành vi của chương trình. Do đó việc thiết kế kiểm thử hàm là việc phân tách đặc tả các hành vi có thể của chương trình thành một số lớp mà mỗi lớp này sẽ có hành vi nhất quán hay tương tự. Với ví dụ hàm giải phương trình bậc hai trên chúng ta có thể chia đặc tả thành hành vi phương trình cho nghiệm kép, hành vi phương trình cho kết quả vô nghiệm, v.v.
- Việc phân tích đặc tả sẽ giúp chúng ta xác định và phân tách được các lớp này. Bên cạnh đó việc phân tích cũng thường giúp chúng ta xác định những điểm không rõ ràng, không đầy đủ của đặc tả. Đây là quá trình phân tích tổng hợp nhiều nguồn thông tin, từ đặc tả, từ kiến thức, kinh nghiệm của người thiết kế kiểm thử, do đó nó mang tính chủ công và cũng dễ mắc lỗi. Nhiều chuyên gia về thiết kế kiểm thử hàm nhiều khi vẫn bỏ sót một số ca kiểm thử quan trọng. Để

giảm thiểu các sai sót này, chúng ta cần có phương pháp hệ thống giúp chia quá trình thiết kế này thành các bước cơ bản để thực hiện. Từ đó một số bước sẽ được tự động hóa để giảm công sức, giảm sai sót, và tăng được chất lượng công việc. Thực tế việc tự động hoàn toàn chưa thực hiện được và một số bước cơ bản vẫn phải thực hiện thủ công. Do đó kỹ năng và kinh nghiệm của người thiết kế kiểm thử đóng vai trò quyết định đến chất lượng của bộ kiểm thử.

- Để hiểu rõ hơn các công việc trong thiết kế kiểm thử hàm, chúng ta giả sử đang thiết kế kiểm thử cho chương trình P và giả sử $Y = P(X)$ trong đó X là vec-tơ của n biến đầu vào và Y là vec-tơ của m biến đầu ra. Dựa trên hiểu biết của chúng ta về đặc tả của chương trình P , với một vec-tơ đầu vào $X = (a_1, \dots, a_n)$ với a_i là các giá trị cụ thể thì chúng ta tính được một vec-tơ kết quả cụ thể $Y = (b_1, \dots, b_m)$. Kết quả Y này là được gọi là *kết quả mong đợi*. Kết quả thực của chương trình P với đầu vào X khi chạy chương trình là $P(X)$. Chúng ta cần kiểm tra kết quả mong đợi đúng bằng kết quả thực của chương trình, tức là Y và $P(x)$ là một hay không. Nếu đúng thì chương trình đã chạy đúng với X . Nếu chương trình chạy đúng với mọi giá trị X như mô tả trong đặc tả thì chúng ta kết luận chương trình P đã được cài đặt đúng so với đặc tả.

• 3.1.1 Phương pháp hệ thống

- Việc xác định các ca kiểm thử từ đặc tả chức năng là quá trình phân tích để xác định và chia không gian đầu vào/đầu ra/hành vi của chương trình thành các miền con. Phương pháp hệ thống sẽ chia quá trình này thành các bước cơ bản, từ đó làm từng bước được dễ dàng hơn hoặc tự động hóa một số bước nếu có thể. Phương pháp hệ thống giúp chúng ta xác định bộ ca kiểm thử đã đủ chưa đồng thời tránh tạo ra nhiều ca kiểm thử trùng lặp, không cần thiết.
- Hình 3.1 mô tả các bước chính của một phương pháp hệ thống. Một số bước trong qui trình này có thể phức tạp, một số bước khá đơn giản và một số bước có thể bỏ qua tùy theo miền ứng dụng và đặc tả chương trình cũng như kinh nghiệm của người thiết kế. Dựa trên qui trình tổng quát này chúng ta có thể đưa ra các qui trình cụ thể bằng việc cụ thể hóa các bước trong qui trình. Chúng ta sẽ xem xét chi tiết hơn các bước trong phương pháp này.



• Hình 3.1: Các bước chính của phương pháp hệ thống cho kiểm thử hàm

- **Xác định các chức năng kiểm thử độc lập được:** Đặc tả phần mềm trên thực tế là lớn và phức tạp ngay cả với những hệ thống nhỏ. Chúng thường được phân rã thành các đơn vị nhỏ hơn như các hệ thống con hay cụ thể hơn là các ca sử dụng (use case) hoặc các kịch bản sử dụng (user stories). Chúng ta gọi chung là các chức năng. Ví dụ trang web tìm kiếm của Google có thể có tính năng tìm kiếm, hiển thị kết quả, hiển thị quảng cáo, hay đăng ký người sử dụng mới, v.v.. Tuy nhiên ranh giới của một chức năng nhiều khi không rõ ràng. Ví dụ chức năng tìm kiếm của Google Search có chức năng tính toán một biểu thức số học nếu văn bản tìm kiếm là một biểu thức số học.
- Khi thiết kế kiểm thử chúng ta tách các chức năng càng chi tiết, cụ thể càng tốt vì nó vừa giúp đơn giản việc thiết kế kiểm thử và vừa cho phép chúng ta xác định lỗi trong chương trình dễ dàng hơn. Mỗi ca kiểm thử chỉ nên kiểm tra một kịch bản cụ thể, chi tiết của một chức năng của hệ thống.
- Tiếp đó với mỗi chức năng chúng ta sẽ xác định tất cả các đầu vào có ảnh hưởng đến việc thực hiện chức năng đó và các đầu ra có thể bị tác động bởi chức năng đó. Các đầu vào và đầu ra thường được mô tả tường minh trong tài liệu đặc tả.

Nhưng cũng có nhiều trường hợp các đầu vào và đầu ra này nằm ẩn trong tài liệu đặc tả mà người thiết kế phải phát hiện ra. Ví dụ như các đặc tả không hình thức mô tả chức năng đăng ký tài khoản mới thường chỉ nêu các trường dữ liệu cần khai báo nhưng không nói gì đến chi tiết kiểu dữ liệu sẽ lưu thông tin hồ sơ người sử dụng. Hay một ví dụ khác khá rắc rối với tên tiếng Việt khi chúng ta chỉ có hai trường họ và tên để nhập vào. Một là thông thường chúng ta không nói ra cụ thể chiều dài tối đa, tối thiểu, hay qui định bảng mã ký tự của các trường này. Hai là chúng ta không nói rõ phần tên đệm sẽ nằm ở phần họ hay phần tên, nên khi người sử dụng nhập dữ liệu sẽ xảy ra tình trạng mỗi người nhập một kiểu. Tên là Nguyễn Văn Hùng thì có người nhập phần họ là Nguyễn, phần tên là Văn Hùng; có người lại nhập họ là Nguyễn Văn, tên là Hùng; lại có người chỉ nhập họ là Nguyễn, tên là Hùng, vì nghĩ phần Văn là tên đệm không hỏi thì không nhập.

- **Xác định các lớp giá trị đại diện:** Các giá trị đại diện có thể được xác định trực tiếp từ đặc tả phi hình thức, như ngôn ngữ tự nhiên. Chúng cũng có thể được xác định gián tiếp dựa trên một mô hình được xây dựng để phục vụ cho việc này. Trong cả hai trường hợp này mục tiêu đều là xác định các giá trị của mỗi đầu vào một cách độc lập, bằng việc lấy các giá trị đã liệt kê tường minh trong đặc tả, hoặc thông qua một mô hình phù hợp. Chúng ta chưa xét đến tổ hợp của các giá trị này. Lý do là chúng ta đang tách bài toán xác định các lớp giá trị của mỗi đầu vào và bài toán tổ hợp chúng để tạo thành các ca kiểm thử. Tức là chúng ta đang tách một bước phức tạp thành các bước đơn giản hơn.
- Với các đặc tả không hình thức người thiết kế kiểm thử phải dựa vào việc liệt kê tường minh các lớp giá trị đại diện. Trong trường hợp này người thiết kế kiểm thử cần cẩn thận xét tất cả các trường hợp có thể và khai thác tối đa những thông tin có sẵn trong đặc tả. Tương tự, với kết quả mong đợi đầu ra, chúng ta cũng có thể xác định các lớp giá trị của miền đầu ra, cũng như các giá trị biên và các giá trị đặc biệt để có lỗi.
- Ví dụ khi xem xét các thao tác trên một danh sách không rỗng chúng ta cần xét cả trường hợp danh sách rỗng (giá trị lỗi) và danh sách có một phần tử (giá trị biên chiều dài danh sách) là các trường hợp đặc biệt. Ở bước này chúng ta chỉ quan tâm đến tính chất của giá trị, chứ chưa cần cụ thể giá trị là gì. Tức là chúng

ta quan tâm đến đặc tả ca kiểm thử chứ chưa phải ca kiểm thử cụ thể. Với ví dụ danh sách một phần tử, chúng ta chưa cần quan tâm phần tử của danh sách này cụ thể là gì, chỉ biết danh sách chỉ có một phần tử.

- Để liệt kê các giá trị không tường minh chúng ta cần xây dựng mô hình hoặc một phần của mô hình của đặc tả. Ví dụ chúng ta có thể dùng văn phạm để mô tả ngôn ngữ và sinh các từ của ngôn ngữ này để liệt kê các giá trị cần xác định. Mô hình này có thể có sẵn trong đặc tả nhưng thông thường người thiết kế kiểm thử phải xây dựng chúng.
- Trong hai cách trên, việc liệt kê trực tiếp có vẻ đơn giản hơn và ít tốn kém hơn việc xây dựng mô hình để sinh ra các giá trị. Tuy nhiên về lâu về dài các mô hình sẽ có giá trị hơn, khi khối lượng giá trị lớn chúng ta có thể tự động hóa việc liệt kê, hay chúng ta cũng có thể điều chỉnh số lượng ca kiểm thử nhiều hay ít để cân bằng với các ràng buộc kiểm thử khác, như chi phí, thời gian. Các mô hình cũng có thể được sử dụng lại, sửa đổi dễ dàng hơn trong quá trình phát triển hệ thống. Quyết định cuối cùng nên dùng cách nào vẫn tùy thuộc vào lĩnh vực ứng dụng, kỹ năng của người thiết kế kiểm thử, và các công cụ thích hợp đang có.
- **Sinh các đặc tả ca kiểm thử:** Đặc tả kiểm thử được tạo ra bằng việc tổ hợp các giá trị của các đầu vào của chương trình đang được kiểm thử. Bước trên chúng ta đã xác định các giá trị đại diện thì ở bước này chúng ta sinh các tổ hợp của chúng bằng tích Đề-các của các giá trị đại diện đã chọn. Nếu ở bước trước chúng ta tạo ra mô hình hình thức thì ở bước này các đặc tả ca kiểm thử sẽ là các hành vi cụ thể hoặc tổ hợp của các tham số của mô hình và một đặc tả ca kiểm thử có thể được thỏa mãn bởi nhiều đầu vào cụ thể. Trong cả hai cách này việc tạo ra tất cả các tổ hợp thường tạo ra số lượng quá lớn các ca kiểm thử.
- Ví dụ một hàm có năm đầu vào, mỗi đầu vào có sáu giá trị đại diện thì tích Đề-các sẽ tạo ra $6^5 = 7,776$ đặc tả ca kiểm thử. Số lượng này là quá lớn với một chức năng, ngay cả khi được tự động hóa và chạy bằng máy tính hoàn toàn. Thông thường, rất nhiều tổ hợp là không hợp lệ hoặc không khả thi. Chúng ta sẽ học các phương pháp để giảm số tổ hợp này nhưng vẫn đảm bảo chất lượng của bộ kiểm thử.
- Để giảm số ca kiểm thử, chúng ta cần loại bỏ các tổ hợp giá trị không hợp lệ bằng

việc đưa thêm các ràng buộc về cách tổ hợp. Ví dụ hàm *NextDate* chúng ta cần đưa ràng buộc để tránh tạo ra các giá trị không hợp lệ như ngày 31/11/2013. Hoặc chúng ta cũng không xét hết tất cả các tích Đề-các mà chỉ cần mỗi cặp giá trị của hai biến bất kỳ đều xuất hiện là đủ. Quá trình này có thể không phải một bước làm là xong mà có thể cần điều chỉnh và thử một số lần để có kết quả là bộ ca kiểm thử phù hợp, vừa hạn chế được các ca kiểm thử vô lý không cần thiết, vừa tiết kiệm được chi phí thực hiện kiểm thử.

- **Sinh ca kiểm thử và thực hiện kiểm thử:** Quá trình kiểm thử hoàn tất khi chúng ta cụ thể hóa các đặc tả kiểm thử thành các ca kiểm thử cụ thể, rồi tính kết quả mong đợi của từng ca kiểm thử, và tiến hành kiểm tra với phần mềm. Việc thực hiện kiểm thử cần được tự động hóa tối đa khi có thể, vì một hệ thống phần mềm cần được kiểm thử rất nhiều lần. Ngay cả khi sản phẩm hoàn tất, trong tương lai chúng ta vẫn sẽ cần thực hiện kiểm thử nhiều lần nữa, khi phần mềm tiến hóa, thay đổi theo những yêu cầu mới, hay các lỗi được phát hiện và phần mềm cần phải sửa.

- **3.1.2 Lựa chọn phương pháp phù hợp**

- Trong các chương sau chúng ta sẽ thấy một số kỹ thuật kiểm thử hàm thích hợp cho các loại đặc tả chương trình khác nhau. Với một đặc tả có thể có một số kỹ thuật phù hợp để xây dựng các ca kiểm thử và một số kỹ thuật sẽ rất khó áp dụng hoặc kết quả sẽ không thỏa mãn. Một số kỹ thuật có thể cho kết quả như nhau với một số đặc tả. Một số kỹ thuật lại bổ sung cho nhau nên cần kết hợp chúng. Ví dụ như khi mỗi kỹ thuật lại áp dụng tốt cho những khía cạnh khác nhau của cùng một đặc tả, hoặc ở các giai đoạn khác nhau của việc sinh ca kiểm thử.
- Việc lựa chọn kỹ thuật để tạo ra các ca kiểm thử phụ thuộc vào các yếu tố: tính tự nhiên của đặc tả, dạng của đặc tả, kinh nghiệm của người thiết kế kiểm thử, cấu trúc của tổ chức, công cụ sẵn có, ngân sách và các ràng buộc về chất lượng, chi phí thiết kế và cài đặt các hệ thống hỗ trợ.
- **Tính tự nhiên và dạng của đặc tả:** Các kỹ thuật kiểm thử khai thác các đặc điểm khác nhau của đặc tả. Ví dụ, khi biến đầu vào có một số ràng buộc thì có thể kỹ thuật kiểm thử lớp tương đương sẽ phù hợp. Nếu có sự chuyển dịch trong các trạng thái của hệ thống thì phương pháp kiểm thử dựa trên máy trạng thái có thể

nên áp dụng. Hay khi các đầu vào thay đổi với kích thước không xác định được thì phương pháp dùng ngữ pháp có thể phù hợp.

- **Kinh nghiệm của người thiết kế kiểm thử:** Kinh nghiệm của người kiểm thử nhiều khi sẽ ảnh hưởng đến kỹ thuật kiểm thử nào được sử dụng. Ví dụ một người thiết kế kiểm thử giỏi về bảng quyết định sẽ thích kỹ thuật đó khi một kỹ thuật khác cũng có thể áp dụng được.
- **Công cụ:** Một số kỹ thuật đòi hỏi sử dụng các công cụ mà đôi khi phải mua nên chi phí cũng là một yếu tố ảnh hưởng đến việc lựa chọn kỹ thuật kiểm thử. Ví dụ đặc tả sử dụng UML và có công cụ thương mại sinh ca kiểm thử dựa trên một số biểu đồ UML thì công cụ đóng vai trò quan trọng ảnh hưởng đến phương pháp thực hiện kiểm thử, nếu ngân sách cho kiểm thử cho phép. Một số đơn vị làm phần mềm chuyên nghiệp đôi khi phải xây dựng các công cụ này để phục vụ cho đơn vị, hoặc đôi khi cho từng dự án cụ thể.
- **Ngân sách và ràng buộc chất lượng:** Ngân sách và ràng buộc chất lượng có thể dẫn đến sự lựa chọn khác nhau về phương pháp kiểm thử. Ví dụ nếu yêu cầu là nhanh, kiểm thử tự động, nhưng không cần độ tin cậy cao thì kiểm thử ngẫu nhiên là phù hợp. Ngược lại, yêu cầu độ tin cậy phải rất cao thì chúng ta phải sử dụng các phương pháp tinh vi để kiểm thử một cách kỹ lưỡng nhất có thể. Khi chọn một phương pháp, điều quan trọng là phải đánh giá tất cả các yếu tố và chi phí liên quan.
- **Chi phí nền tảng kiểm thử:** Mỗi đặc tả ca kiểm thử sẽ phải được cụ thể hóa thành một ca kiểm thử và được chạy đi chạy lại nhiều lần trong suốt vòng đời của sản phẩm. Mỗi lần chạy một ca kiểm thử bao gồm việc phải đưa đầu vào vào chương trình, thực hiện chương trình, và so sánh kết quả đầu ra thực tế với đầu ra mong đợi. Nếu chúng ta tự động hóa được tất cả các công việc này thì phương pháp tổ hợp sinh ra tất cả các tổ hợp có thể áp dụng. Tuy nhiên nếu một số bước phải thực hiện bằng tay, hoặc chúng ta phải viết mã cho từng đặc tả kiểm thử này thì có lẽ chúng ta phải chọn phương pháp khác để khả thi về mặt thời gian và công sức.
- Tóm lại kiểm thử hàm cũng giống nhiều hoạt động kỹ thuật khác đòi hỏi phải phân tích kỹ lưỡng các ưu nhược điểm để chọn phương pháp cân bằng phù hợp

với các khó khăn và các vấn đề không lường hết được mà phải là người thiết kế có kinh nghiệm mới có cảm giác tốt được. Kiểm thử hàm không phải là việc tập để chọn một phương pháp tối ưu mà là một loạt các hoạt động để tìm ra sự tổ hợp của một số mô hình và kỹ thuật để tạo ra một tập các ca kiểm thử thỏa mãn chi phí và ràng buộc chất lượng đặt ra. Sự cân bằng này cũng có thể mở rộng ra ngoài thiết kế kiểm thử sang thiết kế phần mềm để thuận lợi cho việc kiểm thử. Thiết kế phần mềm thích hợp sẽ không chỉ giúp quá trình phát triển được tốt hơn mà còn giúp việc kiểm thử cũng thuận lợi để giúp tiết kiệm chi phí tổng thể của dự án phần mềm.

3.2 Kiểm thử lớp tương đương

Ý tưởng của kiểm thử lớp tương đương là chia miền đầu vào thành các phần sao cho các giá trị trong mỗi phần có tác động tương tự với chương trình. Khi đã chia được thành các miền con như vậy, ta chỉ cần lấy một giá trị bất kỳ trong mỗi miền con để xây dựng bộ kiểm thử. Phương pháp kiểm thử lớp tương đương này cho ta cảm giác đã kiểm thử đầy đủ hệ thống đồng thời cũng không có các ca kiểm thử trùng lặp. Chúng ta cũng có thể áp dụng lớp tương đương cho miền đầu ra.

Kiểm thử lớp tương đương là phương pháp chia miền dữ liệu kiểm thử thành các miền con sao cho dữ liệu trong mỗi miền con có cùng tính chất đối với chương trình, có nghĩa là các ca kiểm thử của một miền con sẽ cùng gây lỗi cho chương trình, hay cùng cho kết quả đúng, hay cùng cho kết quả sai tương tự nhau. Sau khi chia miền dữ liệu của chương trình thành và miền con tương đương, chúng ta chỉ cần chọn một phần tử đại diện của mỗi miền con này làm bộ dữ liệu kiểm thử. Các miền con này chính là các lớp tương đương.

Ví dụ trong bài toán Triangle ta lấy một lớp tương đương là tất cả các tam giác đều. Khi đó ta chọn một ca kiểm thử là (5, 5, 5) để thực hiện thì sau đó ta có thể kết luận tương tự về kết quả kiểm thử cho các ca kiểm thử khác cùng lớp như (6, 6, 6) hay (100, 100, 100). Bản chất ở đây là chúng ta cho rằng chương trình sẽ thực hiện cùng các lệnh giống nhau với các ca kiểm thử cùng một lớp tương đương, nên hành vi của chương trình là tương tự nhau – cùng có lỗi hay ra kết quả sai hay cùng ra các kết quả đúng.

Bây giờ vấn đề chính là làm sao xác định được các lớp tương đương của miền đầu vào. Việc này có thể thực hiện bằng cách phân tích mã nguồn của chương trình. Nếu không phân tích được mã nguồn, chúng ta có thể dựa vào hiểu biết về chức năng của chương trình mà suy ra cách cài đặt của chúng. Từ đó chúng ta có thể suy ra các lớp tương đương theo kinh nghiệm và trực giác của mình.

Kiểm thử lớp tương đương được chia thành một số loại. Chúng ta sẽ sử dụng chương trình minh họa sau để giải thích các loại kiểm thử lớp tương đương dễ dàng hơn.

Giả sử chương trình của chúng ta là một hàm của ba biến a, b, c và không gian đầu vào là ba tập A, B, C . Giả sử tiếp là chúng ta đã xác định được các miền tương đương cho mỗi không gian đầu vào:

$$A = A_1 \cup A_2 \cup A_3$$

$$B = B_1 \cup B_2 \cup B_3 \cup B_4$$

$$C = C_1 \cup C_2$$

Chúng ta ký hiệu phần tử của các miền tương đương trên bằng chữ thường với cùng chỉ số trong các phần sau. Ví dụ $a_1 \in A_1, b_3 \in B_3$ và $c_2 \in C_2$.

3.3.1 Kiểm thử lớp tương đương yếu

Kiểm thử lớp tương đương yếu chỉ yêu cầu mỗi không gian tương đương có ít nhất một phần tử xuất hiện trong một ca kiểm thử nào đó. Với chương trình minh họa trên thì phải có một giá trị thuộc A_1 trong một ca kiểm thử nào đó. Tương tự với các tập con khác. Ta có bộ kiểm thử thỏa mãn kiểm thử lớp tương đương yếu như trong Bảng 3.1.

Bảng 3.1: Các ca kiểm thử lớp tương đương cho Triangle

TT	a	b	c
1	a_1	b_1	c_1
2	a_2	b_2	c_2
3	a_3	b_3	c_3
4	a_1	b_4	c_2

Bộ kiểm thử trên sử dụng một giá trị từ mỗi lớp tương đương. Để kiểm tra chúng ta chỉ cần để ý đến các cột, ví dụ cột a có xuất hiện cả a_1, a_2, a_3 của ba miền con của

A , như vậy là đủ.

Chúng ta có thể lập qui tắc để tự động xây dựng bộ kiểm thử lớp tương đương yếu này dễ dàng và cũng có thể dễ nhận thấy là số ca kiểm thử tối thiểu chính là số lớp tương đương lớn nhất của các miền đầu vào, trong ví dụ này là B .

3.3.2 Kiểm thử lớp tương đương mạnh

Kiểm thử lớp tương đương mạnh sẽ kết hợp các tổ hợp có thể của các miền tương đương. Với ví dụ trên ta sẽ có $3 \times 4 \times 2 = 24$ tổ hợp tương ứng với 24 ca kiểm thử như bảng 3.2.

Việc xác định các ca kiểm thử ở bảng trên tương tự như việc xây dựng bảng giá trị chân lý cho logic mệnh đề nên cũng dễ dàng tự động hóa được. Tích Đề-các này cho ta một bộ kiểm thử đầy đủ, có tất cả các khả năng kết hợp của các miền tương đương của các miền đầu vào.

Ở hai phần trên chúng ta chỉ chú trọng các miền tương đương của biến đầu vào. Chúng ta có thể áp dụng các phương pháp này cho miền đầu ra. Việc này thường khó hơn vì chúng ta phải tính ngược bộ giá trị kiểm thử ở miền đầu vào để chúng tạo ra giá trị ở miền đầu ra mong muốn. Với chương trình có nhiều biến đầu ra, chúng ta cũng có hai dạng mạnh và yếu tương tự với đầu vào.

Bảng 3.2: Các ca kiểm thử lớp tương đương mạnh cho Triangle

TT	a	b	c
1	a_1	b_1	c_1
2	a_1	b_1	c_2
3	a_1	b_2	c_1
4	a_1	b_2	c_2
5	a_1	b_3	c_1
6	a_1	b_3	c_2
7	a_1	b_4	c_1
8	a_1	b_4	c_2
9	a_2	b_1	c_1
10	a_2	b_1	c_2
11	a_2	b_2	c_1
12	a_2	b_2	c_2
13	a_2	b_3	c_1
14	a_2	b_3	c_2
15	a_2	b_4	c_1
16	a_2	b_4	c_2
17	a_3	b_1	c_1
18	a_3	b_1	c_2

19	a_3	b_2	c_1
20	a_3	b_2	c_2
21	a_3	b_3	c_1
22	a_3	b_3	c_2
23	a_3	b_4	c_1
24	a_3	b_4	c_2

3.3.3 Kiểm thử lớp tương đương đơn giản

Kiểm thử lớp tương đương đơn giản chỉ phân chia một lớp gồm các giá trị hợp lệ và các miền giá trị không hợp lệ. Ví dụ nếu A là khoảng 1 đến 200 các số nguyên thì miền hợp lệ là các giá trị từ 1 đến 200, miền không hợp lệ gồm hai miền. Miền thứ nhất là các giá trị từ 0 trở xuống. Miền còn lại là các giá trị từ 201 trở lên. (Để đơn giản ta không xét miền các giá trị không phải là số nguyên.) Sau khi đã có các lớp tương đương theo cách đơn giản này chiến lược kiểm thử có thể áp dụng tương tự kiểm thử tương đương mạnh và yếu.

Có hai vấn đề với kiểm thử lớp tương đương đơn giản. Thứ nhất là thông thường khi mô tả bài toán thì tài liệu không nói rõ các trường hợp khi đầu vào không hợp lệ thì kết quả là gì. Nếu có nói thì cũng thường chỉ đề cập đến một số trường hợp đã biết mà không xét đến mọi khả năng có thể. Ví dụ chúng ta viết chương trình giải phương trình bậc hai với ba hệ số nguyên a , b , c được nhập vào từ bàn phím. Rất nhiều lập trình viên sẽ chuyển ngay xâu ký tự người sử dụng nhập vào thành các số và gán cho các hệ số này. Nhưng nếu người sử dụng nhập một xâu chữ thì chương trình không kiểm tra để có thông báo phù hợp cho người sử dụng. Khi thiết kế kiểm thử chúng ta phải bổ sung một loạt các trường hợp mà hệ thống phải đưa ra kết quả bổ sung hoặc các thông báo lỗi phù hợp cho các trường hợp này. Thứ hai là một số ngôn ngữ lập trình có hệ thống kiểm tra kiểu mạnh như Java, C# thì không thể đưa giá trị sai kiểu vào được nên chúng ta không cần các ca kiểm thử cho giá trị sai kiểu, nhưng vẫn cần kiểm tra dữ liệu do người sử dụng nhập vào, và chỉ kiểm tra dữ liệu không hợp lệ cùng kiểu. Tuy nhiên trong các ngôn ngữ lập trình script như Javascript, PHP, Python, không có kiểm tra kiểu mạnh nên các lỗi loại này khá phổ biến do đó ta cần xét kỹ việc chia miền đầu vào thành các lớp tương đương.

❖ Kinh nghiệm áp dụng

Từ các ví dụ trên chúng ta có một số quan sát và định hướng sử dụng kiểm thử

lớp tương đương.

- Kiểm thử lớp tương đương đơn giản kém kiểm thử lớp tương đương yếu kém kiểm thử lớp tương đương mạnh.

- Sử dụng kiểm thử tương đương đơn giản khi ngôn ngữ không có kiểm tra kiểu mạnh.

- Nếu cần kiểm tra ngoại lệ chúng ta nên mở rộng kiểm thử lớp tương đương với các lớp giá trị ngoài miền xác định.

- Kiểm thử lớp tương đương phù hợp với dữ liệu đầu vào có miền giá trị là khoảng và hữu hạn.

- Kiểm thử lớp tương đương kết hợp với kiểm thử giá trị biên sẽ tốt hơn.

- Nên dùng kiểm thử lớp tương đương với các chương trình phức tạp.

- Nên dùng kiểm thử lớp tương đương mạnh khi các biến là độc lập. Khi các biến phụ thuộc nhau thì dễ tạo ra các ca kiểm thử vô lý.

Câu hỏi cuối bài:

1. Nêu ý tưởng phương pháp kiểm thử lớp tương đương?

2. Nguyên tắc phân hoạch lớp tương đương?

3. Các bước thực hiện kiểm thử lớp tương đương?

4. Phân loại kiểm thử lớp tương đương?

5. Các bài tập thực hành:

Bài 1. Bài tập kiểm thử lớp tương đương với số tuổi

Một ứng dụng yêu cầu người dùng nhập số tuổi để xác minh điều kiện tham gia, với điều kiện:

- Tuổi phải nằm trong khoảng từ **18 đến 60** (bao gồm cả 18 và 60).

Yêu cầu:

- (a) Xác định các lớp tương đương (hợp lệ và không hợp lệ).

- (b) Tạo test case cho từng lớp tương đương.

Bài 2. Bài tập kiểm thử lớp tương đương với điểm thi

Một hệ thống quản lý điểm yêu cầu nhập điểm môn học, với quy định:

- Điểm phải nằm trong khoảng từ **0 đến 100**.

Yêu cầu:

- (a) Xác định các lớp tương đương cho giá trị điểm nhập vào.
- (b) Viết các test case đại diện cho từng lớp tương đương.

Bài 3. Bài tập kiểm thử lớp tương đương với mật khẩu

Hệ thống yêu cầu người dùng nhập mật khẩu, với quy tắc:

- Độ dài mật khẩu phải từ **8 đến 16 ký tự**.
- Chỉ chứa các ký tự chữ cái và số (không được chứa ký tự đặc biệt).

Yêu cầu:

- (a) Xác định các lớp tương đương cho độ dài và nội dung của mật khẩu.
- (b) Viết các test case đại diện cho các lớp tương đương đã xác định.

Bài 4. Bài tập kiểm thử lớp tương đương với giao dịch ngân hàng

Hệ thống giao dịch ngân hàng yêu cầu người dùng nhập số tiền giao dịch, với các quy định:

- Số tiền tối thiểu: **100.000 VND**.
- Số tiền tối đa: **50.000.000 VND**.

Yêu cầu:

- (a) Xác định các lớp tương đương cho số tiền nhập vào.
- (b) Tạo test case cho các giá trị trong và ngoài các lớp tương đương.

Bài 5. Bài tập kiểm thử lớp tương đương với hệ thống đăng ký vé máy bay

Một hệ thống đặt vé máy bay yêu cầu nhập các thông tin sau:

- Số lượng vé: **1 đến 5 vé**.
- Tổng số hành lý: **0 đến 30 kg**.

Yêu cầu:

- (a) Xác định các lớp tương đương cho từng trường dữ liệu.
- (b) Tạo test case đại diện cho các lớp tương đương hợp lệ và không hợp lệ.